

Problem 11 – Smart Restaurant Ordering System 🍔

Difficulty: ★★☆☆ Beginner+

1. Overview

Build a **Smart Restaurant Ordering System** using **Python** and **Streamlit**.

The customer should be able to:

- Enter their name.
- Choose a main meal.
- Choose a drink.
- Optionally choose a dessert.
- Place an order.
- Receive a detailed bill.
- Get a 10% discount when eligible.

The restaurant should also keep track of all orders and generate a daily sales report.

🚨 **Important:** This project must be built using **Streamlit only**.
Do not use console input, console output, or a console menu.

2. Restaurant Menu

Store the menu using **three dictionaries**.

Main Meals

Meal	Price (EGP)
------	-------------

Burger	180
--------	-----

Pizza	220
-------	-----

Pasta	170
-------	-----

Fried Chicken	200
---------------	-----

Drinks

Drink	Price (EGP)
-------	-------------

Cola	35
------	----

Orange Juice	45
--------------	----

Water	20
-------	----

Coffee	50
--------	----

Desserts

Dessert	Price (EGP)
---------	-------------

Ice Cream	60
-----------	----

Cake	70
------	----

Donut	45
-------	----

3. Application Layout

The application must contain:

Sidebar

Display:

- Today's Special
- Opening Hours
- Short restaurant information

The complete restaurant menu should **not** be displayed in the sidebar.

Three Tabs

Create:

1. **New Order**
 2. **Menu**
 3. **Bill**
-

4. New Order Tab

Create an ordering form.

The form must allow the customer to enter:

Customer Name

Use `st.text_input()`.

Main Meal

Use `st.selectbox()` and display the available meals.

Drink

Use `st.selectbox()` and display the available drinks.

Dessert

Use `st.selectbox()`.

Include **No Dessert** as an option because dessert is optional.

Place Order

Use `st.form_submit_button()`. **All ordering controls must be inside an `st.form()`.**

5. Validate the Order

When the customer submits the form:

- The customer name must not be empty.
- The selected meal must be valid.
- The selected drink must be valid.
- The dessert must be a valid dessert or **No Dessert**.

If the customer name is empty, display an error message using `st.error()`.

If **No Dessert** is selected, display a warning using `st.warning()`.

The order should still be valid when no dessert is selected.

6. Calculate the Bill

Calculate the original total:

Original Total = Meal + Drink + Dessert

If the customer chooses **No Dessert**, the dessert price is zero.

Discount

If the original total is **300 EGP or more**, give the customer a **10% discount**.

Otherwise, there is no discount.

Calculate:

- Original Total
- Discount
- Final Total

Example

Item	Price
Pizza	220 EGP
Cola	35 EGP
Cake	70 EGP

Original Total 325 EGP

Discount 32.5 EGP

Final Total 292.5 EGP

7. After Placing an Order

After a valid order:

- Save the order.
 - Display a success message using `st.success()`.
 - Use `st.spinner()` to simulate preparing the order.
 - Make the order available in the Bill tab.
-

8. Menu Tab

The **Menu tab is the only place where the complete restaurant menu is displayed.**

Organize the menu into:

Main Meals

Display all meals and prices.

Drinks

Display all drinks and prices.

Desserts

Display all desserts and prices.

Use the dictionaries to generate the menu rather than repeating the menu data manually.

9. Bill Tab

The Bill tab displays the **most recently placed order.**

If no order has been placed yet, display an information message using `st.info()`.

Use `st.columns()` to create **two columns.**

Left Column – Ordered Items

Display:

- Customer name
- Meal
- Drink
- Dessert

Right Column – Prices

Display:

- Meal price
- Drink price
- Dessert price
- Original Total
- Discount
- Final Total

Use `st.code()` to display a formatted **Order Summary**.

10. Required Text Functions

You must use **all six** functions below.

Function	Required Use
<code>st.title()</code>	Application title
<code>st.header()</code>	Major sections
<code>st.subheader()</code>	Smaller sections
<code>st.write()</code>	General information
<code>st.code()</code>	Order summary
<code>st.divider()</code>	Separate sections

11. Required Input Widgets

You must use:

Widget	Purpose
<code>st.text_input()</code>	Customer name
<code>st.selectbox()</code>	Main meal
<code>st.selectbox()</code>	Drink
<code>st.selectbox()</code>	Dessert
<code>st.form_submit_button()</code>	Place order

All of these ordering controls must be inside an `st.form()`.

Do not use a normal `st.button()` as the form submission button.

12. Required Status Elements

You must use **all five**:

Function	Purpose
----------	---------

st.info()	Restaurant information / no order
-----------	-----------------------------------

st.success()	Successful order
--------------	------------------

st.warning()	No dessert selected
--------------	---------------------

st.error()	Invalid input
------------	---------------

st.spinner()	Preparing the order
--------------	---------------------

13. Store Orders

Every successfully placed order must be stored in a **list of dictionaries**.

Each order should contain at least:

- Customer name
- Meal
- Drink
- Dessert
- Original total
- Discount
- Final total

Use `st.session_state` to keep the orders available when Streamlit reruns the application.

The stored orders must not disappear when:

- Switching between tabs.
 - Placing another order.
 - Interacting with the application.
-

14. Required Functions

Create at least these functions:

display_menu()

Responsible for displaying the complete restaurant menu.

calculate_total()

Responsible for calculating:

- Original Total
- Discount
- Final Total

display_bill()

Responsible for displaying the latest order and its bill.

You may create additional functions if needed.

15. Challenge Mission ★★

Daily Sales Report

Create a **Daily Sales Report** using the stored orders.

Display:

Total Orders

The total number of successfully placed orders.

Total Sales

The total of all customers' **final totals**.

Most Ordered Meal

The meal that appears in the greatest number of orders.

Most Ordered Drink

The drink that appears in the greatest number of orders.

Most Ordered Dessert

The dessert that appears in the greatest number of orders.

Ignore **No Dessert**.

Top Customer

The customer who spent the most money.

If a customer placed multiple orders, combine their spending.

Average Order Value

Calculate:

Average Order Value = Total Sales ÷ Number of Orders

Handle the case where there are no orders so that you do not divide by zero.

16. Required Streamlit Topics

Displaying Text

- `st.title()`
- `st.header()`
- `st.subheader()`
- `st.write()`
- `st.code()`
- `st.divider()`

Input Widgets

- `st.text_input()`
- `st.selectbox()`
- `st.form()`
- `st.form_submit_button()`

Status Elements

- `st.info()`
- `st.success()`
- `st.warning()`
- `st.error()`
- `st.spinner()`

Layout

- `st.sidebar`
- `st.tabs()`
- `st.columns()`

State Management

- `st.session_state`
-

17. Python Topics

This project practices:

- Variables
 - Strings
 - Lists
 - Dictionaries
 - Integers
 - Floats
 - Arithmetic operators
 - Comparison operators
 - Assignment operators
 - Membership operators
 - if / elif / else
 - for loops
 - User-defined functions
 - Dictionary methods
 - List operations
-

18. Final Requirements Checklist

Before submitting, make sure your application has:

Restaurant

- Three menu dictionaries
- Sidebar
- Today's Special
- Opening Hours
- Restaurant information

Navigation

- New Order tab
- Menu tab
- Bill tab

Ordering

- Customer name
- Meal selection
- Drink selection
- Optional dessert
- Place Order form
- Input validation

Billing

- Original Total
- 10% discount for totals $\geq$ 300 EGP
- Final Total
- Two-column bill
- Order Summary

Status Elements

- `st.info()`
- `st.success()`
- `st.warning()`
- `st.error()`
- `st.spinner()`

Required Text Functions

- `st.title()`
- `st.header()`
- `st.subheader()`
- `st.write()`
- `st.code()`
- `st.divider()`

Data

- Orders stored in `st.session_state`
- List of dictionaries
- Multiple orders supported

Challenge

- Total orders
- Total sales
- Most ordered meal
- Most ordered drink
- Most ordered dessert
- Top customer
- Average order value